



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

一、Bridge 的核心意图 (Intent)

将抽象部分与实现部分分离，使它们可以独立变化。

注意三个关键词：

- 抽象部分
- 实现部分
- 独立变化

Bridge 不是“抽象硬件”，
而是解决“两个维度同时变化”的问题。

二、嵌入式场景中的问题 (Motivation)

以通信系统为例。

在一个嵌入式产品中，通常存在：

维度 A：协议类型（逻辑层）

- Modbus
- Bootloader 协议
- 私有业务协议
- CANopen

维度 B：传输方式（物理层）

- UART
- TCP
- CAN
- USB

这两个维度是正交的。

如果不使用 Bridge

我们往往会写成：

- Modbus over UART
- Modbus over TCP
- Bootloader over UART
- Bootloader over USB

结构上变成：

协议 × 传输方式

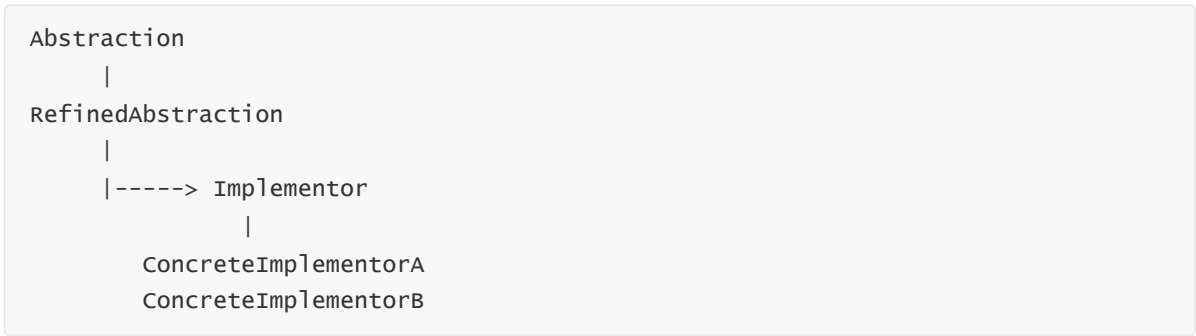
当协议数量为 M，传输方式为 N：

组合复杂度 = $M \times N$

这就是 GoF 书中所谓的“类爆炸问题”。

三、Bridge 的结构思想（Structure）

GoF 结构：



映射到嵌入式通信：



关键点：

- 协议层持有“传输接口”
- 协议不依赖具体 UART / TCP
- 传输层也不依赖具体协议

四、Bridge 在这里解决了什么？

1 把两个变化维度拆开

原本耦合在一起的：

协议 + 传输方式

被拆为：

- 协议层次结构
- 传输层次结构

两个层次结构通过“桥”连接。

2 从 $M \times N$ 变成 $M+N$

新增一个协议：

- 不改任何传输层代码

新增一个传输方式：

- 不改任何协议代码

这才是 Bridge 的核心价值。

五、Bridge 的本质理解（嵌入式角度）

在嵌入式中，Bridge 本质是：

业务逻辑层 与 硬件实现层 的结构性解耦
且双方都可能扩展

它不同于：

- HAL（单一维度抽象）

- Strategy（单一算法替换）

Bridge 一定是**两个维度**。

六、与工厂方法的关系

如果你前面已经讲过工厂方法，可以这样衔接：

- Factory 解决：创建哪个传输实现？
- Bridge 解决：协议如何与传输解耦？

一句话概括：

Factory 决定“用谁”
Bridge 决定“怎么连”

七、Bridge 在嵌入式中的典型应用类别

除了通信协议，还有：

场景	两个变化维度
文件系统	文件 API × 存储介质
GUI 系统	绘图逻辑 × 屏幕驱动
网络协议栈	协议 × 网卡
设备框架	设备抽象 × 具体芯片

但通信例子最清晰。

八、什么时候适合使用 Bridge？

当满足三个条件时：

1. 存在两个独立变化维度
2. 可能形成组合爆炸
3. 两个维度都需要独立扩展

如果只是单一抽象接口：

那更可能是 Strategy 或简单多态。

九、总结

在嵌入式系统中，当我们发现
协议种类在增加，
传输方式也在增加，
且两者相互独立时，

就应该考虑桥接模式。

它的目的不是“抽象硬件”，
而是控制系统结构复杂度。